FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

PRÁCTICA · FRONTEND

Flujo de contratación digital de seguros

Seguros · Seguro de Celular — HTML + Tailwind CSS + Basecoat UI + JavaScript vanilla

Dato	Detalle
Nivel	Junior (HTML, CSS y JavaScript básico)
Modalidad	Individual
Stack	HTML5 · Tailwind CSS v4 · Basecoat UI · JavaScript (sin frameworks)
Diseño base	Figma — páginas "Mobile · 390" y "Desktop · 1440"
Entrega	Repositorio en GitHub + sitio publicado + README con capturas

Link al diseño: https://www.figma.com/design/vJrfOUfiOI7OSeTtRfCloh

FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

1. Objetivo y contexto

Vas a construir el **front-end de un checkout de contratación de seguros**: el usuario elige un producto, cotiza, elige un plan, carga sus datos, paga (simulado) y recibe su número de póliza. Es el mismo tipo de flujo que usan aseguradoras, bancos y telcos para vender productos digitales.

El foco **no** está en la lógica de negocio compleja sino en:

- Maquetar con fidelidad un diseño de Figma usando un sistema de componentes (Basecoat UI).
- Manejar el **estado** de un flujo multipaso con un único objeto JavaScript.
- Validar formularios y dar feedback claro al usuario (errores en línea, botones deshabilitados, estados de carga).
- Resolver el diseño en **mobile y desktop** con el mismo código (responsive).

> **Alcance acotado a propósito**
> No hay backend, base de datos ni pagos reales. Los planes salen de un JSON local y el pago es una función que simula la respuesta según el número de tarjeta. La complejidad está en el flujo y la experiencia, no en el servidor.

2. Recursos

Recurso	Para qué sirve
Figma · página "Mobile · 390"	7 pantallas mobile del flujo + componentes (Button, Input, Stepper) y tokens de color/tipografía.
Figma · página "Desktop · 1440"	7 vistas desktop (checkout en 2 columnas con resumen lateral) + tabla de mapeo de componentes.
Basecoat UI (basecoatui.com)	Componentes con estética shadcn/ui en HTML + Tailwind, sin React.
Tailwind CSS v4 (tailwindcss.com)	Utilidades para layout, espaciado y responsive.
MDN Web Docs	Referencia de DOM, eventos, fetch, validación y accesibilidad.

Figma: https://www.figma.com/design/vJrfOUfiOI7OSeTtRfCloh

Basecoat UI: https://basecoatui.com/installation/

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

> **Cómo usar el Figma**
> Abrí el archivo en modo **Dev Mode** o inspeccioná cada capa para ver tamaños, colores y espaciados. Las pantallas están armadas con auto-layout: el padding y el gap de cada frame equivalen a `p-*` y `gap-*` de Tailwind.
>
> Los textos de ejemplo (Juan Pérez, Galaxy A55, $ 3.000) son datos de prueba: en tu app tienen que salir del estado, no estar escritos a mano en el HTML.

3. Stack y configuración

3.1 Reglas

- HTML, Tailwind CSS v4, Basecoat UI y JavaScript **vanilla**. No se permiten React, Vue, jQuery ni librerías de validación.
- Una sola página (`index.html`). Cada paso es una `<section>` que se muestra u oculta.
- El JavaScript va en archivos separados (nada de lógica en atributos `onclick`).

3.2 Opción recomendada: sin build (CDN)

Es la forma más rápida de arrancar: no requiere Node ni compilación.

```
<!-- index.html -->
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Cubrí Seguros</title>
  <!-- Basecoat UI (estilos de componentes) -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/basecoat-css@1.1.0/dist/basecoat.cdn.min.css">
  <!-- Tailwind v4 en el navegador (utilidades) -->
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  <link rel="stylesheet" href="css/tokens.css">
  <script type="module" src="js/app.js"></script>
</head>
```

3.3 Opción avanzada: Vite + npm (bonus)

```
npm create vite@latest cubri-seguros -- --template vanilla
cd cubri-seguros
npm install tailwindcss @tailwindcss/vite basecoat-css

/* src/style.css */
@import "tailwindcss";
@import "basecoat-css";
```

Para esta práctica **no hace falta el JavaScript de Basecoat**: todos los componentes que usamos (botón, input, card, badge, alert) son solo CSS. Si en el bonus usan toasts, cargan únicamente el JS de ese componente.

3.4 Tokens del diseño

Basecoat expone variables CSS (estilo shadcn). Sobrescribí el color primario para que coincida con el Figma:

```
/* css/tokens.css */
:root {
  --primary: #4F46E5;            /* indigo-600 */
  --primary-foreground: #FFFFFF;
  --radius: 0.75rem;
}
```

Token	HEX	Tailwind	Uso
primary	#4F46E5	indigo-600	Botones, selección, paso activo
primary-700	#3730A3	indigo-800	Textos destacados
primary-50	#EEF2FF	indigo-50	Fondos de selección y resumen
success	#059669	emerald-600	Pasos completos, éxito
error	#DC2626	red-600	Errores de validación y pago
text	#0F172A	slate-900	Texto principal
muted	#64748B	slate-500	Textos secundarios
border	#E2E8F0	slate-200	Bordes
bg	#F5F7FB	—	Fondo de página

Tipografía: **Inter** (Google Fonts). Escala: H2 24/800 · lead 15/400 · label 13/600 · help 12/400.

4. Pantallas y criterios de aceptación

Cada pantalla existe en mobile y en desktop. Los criterios de aceptación son los que se van a verificar al corregir.

Pantalla	Qué tiene que hacer (criterios de aceptación)
01 · Inicio / Catálogo	Muestra 4 productos (Celular, Hogar, Bicicleta, Viaje). Solo **Celular** está habilitado; los demás muestran "Próximamente". El CTA lleva al Paso 1.
02 · Cotizar (Paso 1)	Campos: marca (select), modelo, valor del equipo, antigüedad (opción única). "Ver planes" está deshabilitado hasta que todo sea válido. Errores en línea al salir de cada campo (`blur`).

Pantalla	Qué tiene que hacer (criterios de aceptación)
03 · Elegir plan (Paso 2)	Muestra Básico, Plus y Premium con la cuota calculada según la fórmula de la sección 5. Plus viene preseleccionado con badge "Recomendado". Solo un plan puede estar elegido. El resumen (barra inferior en mobile, card lateral en desktop) se actualiza al cambiar de plan.
04 · Datos personales (Paso 3)	Nombre, apellido, DNI, fecha de nacimiento, email, celular y checkbox de términos. Cada campo inválido muestra su mensaje y el borde en rojo. "Continuar al pago" deshabilitado hasta que todo sea válido.
05 · Pago (Paso 4)	Resumen final generado desde el estado (seguro, equipo, plan, titular, total). Medio de pago (crédito/débito), número de tarjeta, vencimiento, CVV y titular. Al pagar: botón en "Procesando…" deshabilitado durante 1,5 s.
06 · Éxito	Muestra el nombre del titular, email, número de póliza y detalle. "Volver al inicio" reinicia la solicitud.
07 · Error de pago	Mensaje claro y botones "Reintentar pago" y "Cambiar medio de pago". Vuelve al Paso 4 **conservando** todos los datos cargados.
Todas	El stepper refleja el paso actual (completo / activo / pendiente). "Volver" lleva al paso anterior sin perder datos.

5. Reglas de negocio

5.1 Cálculo de la cuota

```
cuota = Math.round(valor × tasaAnual / 12 × factorAntiguedad)
```

Plan	Tasa anual	Coberturas
Básico	4%	Robo con violencia
Plus	6%	Robo con violencia + rotura de pantalla
Premium	9%	Robo, rotura total y hurto/pérdida

Antigüedad	Factor
Menos de 1 año	1,00

Antigüedad	Factor
1 a 2 años	1,10
Más de 2 años	1,25

Casos de prueba (tienen que dar exactamente esto)
Valor $ 600.000 · menos de 1 año → Básico **$ 2.000** · Plus **$ 3.000** · Premium **$ 4.500**
Valor $ 600.000 · 1 a 2 años → Básico **$ 2.200** · Plus **$ 3.300** · Premium **$ 4.950**
Valor $ 1.200.000 · más de 2 años → Básico **$ 5.000** · Plus **$ 7.500** · Premium **$ 11.250**

Los planes viven en `data/planes.json` y se cargan con `fetch`:

```
[
  { "id": "basico",  "nombre": "Básico",  "tasaAnual": 0.04,
    "coberturas": ["Robo con violencia"] },
  { "id": "plus",    "nombre": "Plus",    "tasaAnual": 0.06, "recomendado": true,
    "coberturas": ["Robo con violencia", "Rotura de pantalla"] },
  { "id": "premium", "nombre": "Premium", "tasaAnual": 0.09,
    "coberturas": ["Robo con violencia", "Rotura total", "Hurto / pérdida"] }
]
```

5.2 Validaciones

Campo	Regla	Mensaje de error
Marca	Obligatorio	Elegí una marca
Modelo	2 a 40 caracteres	Ingresá el modelo del equipo
Valor del equipo	Número entre 100.000 y 2.500.000	El valor debe estar entre $100.000 y $2.500.000
Antigüedad	Una opción elegida	Indicá la antigüedad
Nombre / Apellido	Mínimo 2 letras	Ingresá tu nombre / apellido
DNI	`/^\d{7,8}$/`	El DNI debe tener 7 u 8 números
Fecha de nacimiento	Mayor de 18 años	Tenés que ser mayor de edad
Email	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`	Ingresá un email válido
Celular	10 dígitos sin 0 ni 15 (se ignoran espacios y guiones)	Ej.: 11 5555-1234
Términos	Tildado	Tenés que aceptar los términos

Campo	Regla	Mensaje de error
Tarjeta	16 dígitos (se ignoran espacios)	Número de tarjeta inválido
Vencimiento	MM/AA, mes 01–12 y fecha futura	Vencimiento inválido
CVV	3 dígitos	CVV inválido
Titular	Obligatorio	Ingresá el nombre del titular

5.3 Pago simulado y póliza

Número de tarjeta	Resultado
4111 1111 1111 1111	Aprobado → pantalla de éxito
4000 0000 0000 0002	Rechazado → pantalla de error
Cualquier otro número válido	Aprobado

```
export function simularPago(numero) {
  return new Promise((resolve) => {
    setTimeout(() => {
      const limpio = numero.replace(/\s/g, '');
      resolve(limpio === '4000000000000002'
        ? { ok: false, motivo: 'Tarjeta rechazada' }
        : { ok: true });
    }, 1500);
  });
}

// Número de póliza: POL-AAAA-NNNNNN
const poliza = `POL-${new Date().getFullYear()}-${String(contador).padStart(6, '0')}`;
```

> **Importante**
> Nunca uses datos de tarjetas reales, ni siquiera para probar. Usá solo las tarjetas de esta tabla.

6. Arquitectura sugerida

6.1 Estructura de carpetas

```
cubri-seguros/
├── index.html
├── css/
│   └── tokens.css
├── data/
│   └── planes.json
├── js/
│   ├── app.js          ← arranque, eventos y navegación
```

```
│   ├── estado.js       ← objeto solicitud + funciones para modificarlo
│   ├── render.js       ← pinta cada paso a partir del estado
│   ├── validaciones.js ← funciones puras: reciben un valor y devuelven error o null
│   ├── cotizador.js    ← calcularCuota()
│   └── pago.js         ← simularPago() y generarPoliza()
└── README.md
```

6.2 Un único estado

Toda la información del flujo vive en **un solo objeto**. El DOM solo muestra lo que dice el estado: nunca leas "el precio" de un `<span>` para calcular algo.

```
// js/estado.js
export const solicitud = {
  paso: 0,                // 0 = inicio, 1..4 = checkout, 5 = éxito, 6 = error
  equipo:  { marca: '', modelo: '', valor: 0, antiguedad: '' },
  plan: 'plus',
  titular: { nombre: '', apellido: '', dni: '', nacimiento: '',
             email: '', celular: '', terminos: false },
  pago:    { metodo: 'credito', tarjeta: '', vencimiento: '', cvv: '', titular: '' },
  poliza: null,
};

export function irAPaso(n) {
  solicitud.paso = n;
  render();               // siempre después de cambiar el estado
}
```

6.3 Render: mostrar u ocultar pasos

```
<!-- index.html -->
<section data-paso="1" hidden> ... </section>
<section data-paso="2" hidden> ... </section>

// js/render.js
export function render() {
  document.querySelectorAll('[data-paso]').forEach((s) => {
    s.hidden = Number(s.dataset.paso) !== solicitud.paso;
  });
  renderStepper();
  renderResumen();
}
```

6.4 Diagrama de flujo

Navegación entre pantallas y puntos de decisión. Las líneas punteadas son caminos de vuelta.

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

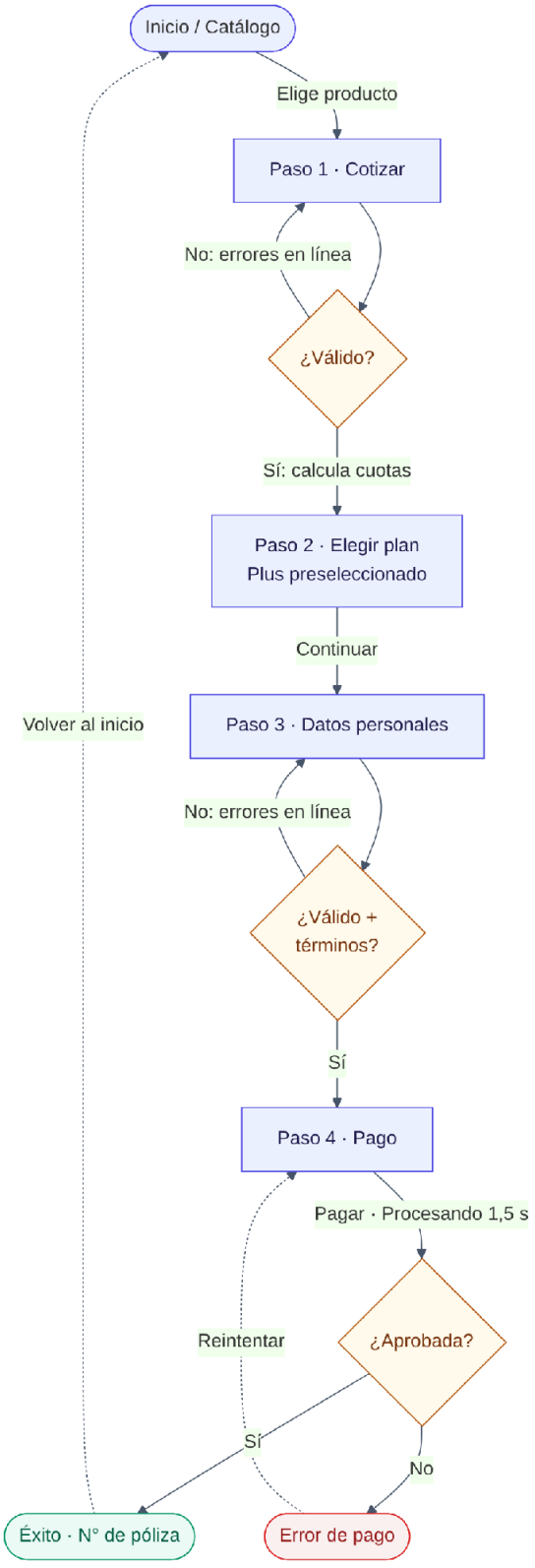


Figura 1 · Flujo de contratación: validaciones por paso y resultado del pago.

FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

6.5 Diagrama de secuencia

Qué pasa entre el usuario, el DOM, `app.js`, el estado, el JSON de planes y el pago simulado, de punta a punta.

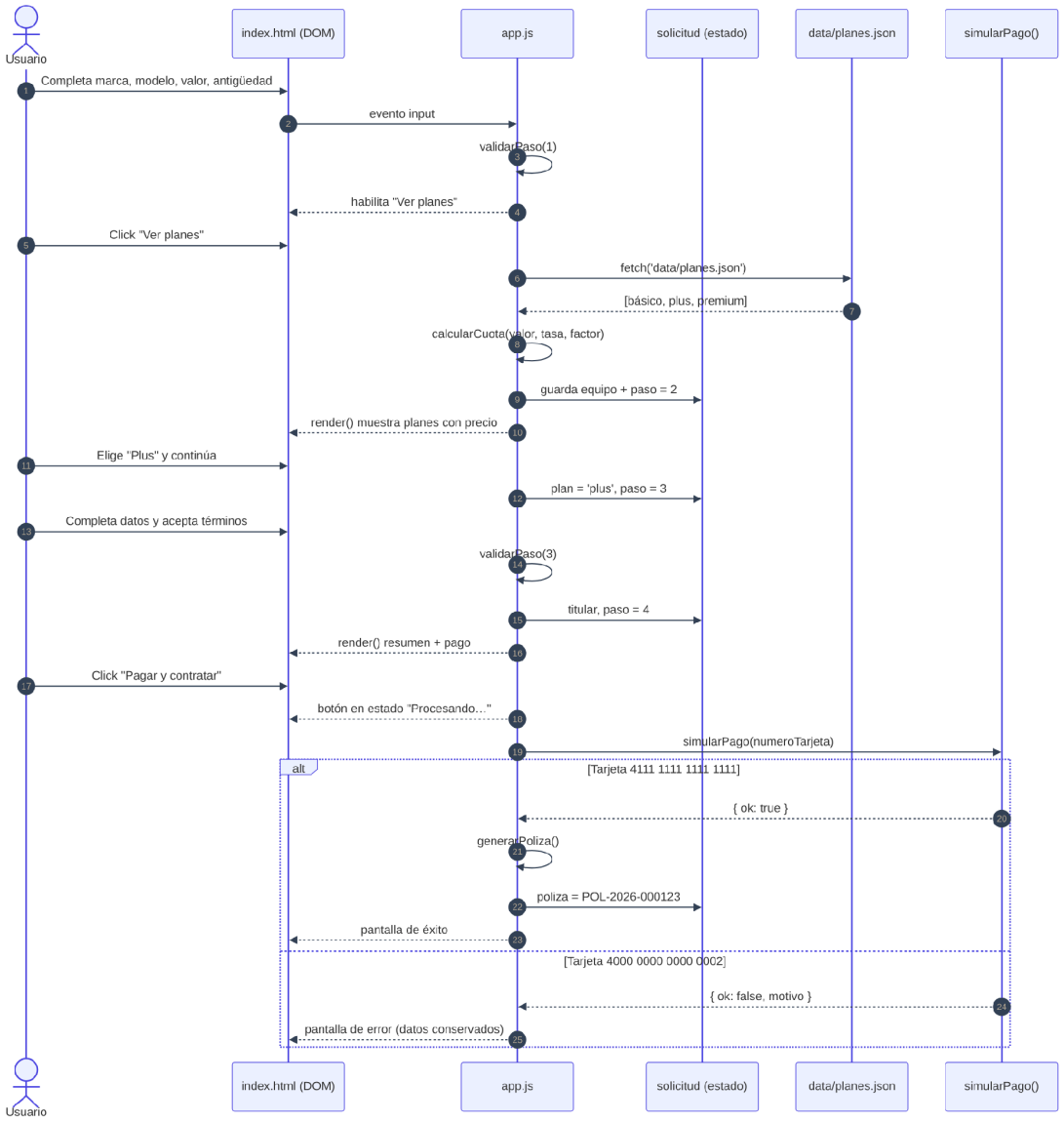


Figura 2 · Secuencia completa, incluyendo los dos resultados posibles del pago.

7. Del Figma a Basecoat UI

Usá los componentes de Basecoat siempre que existan. Lo que no existe (stepper, cards de plan) se arma con utilidades de Tailwind.

Elemento en Figma	Basecoat / HTML	Notas
Button Primary	`<button class="btn">`	Sin `data-variant` = primario
Button Secondary	`<button class="btn" data-variant="outline">`	Botón "Volver"
Button Ghost	`<button class="btn" data-variant="ghost">`	"Volver al inicio"
Button Disabled	`<button class="btn" disabled>`	Se habilita por JS
Input + label + ayuda	`<div role="group" class="field">` con `<label>`, `<input>` y `<p>`	El `<p>` es el texto de ayuda
Input con error	`.field` con `data-invalid` + `aria-invalid="true"` en el input	Mensaje con `role="alert"` y `aria-describedby`
Select (marca)	`<select>` nativo dentro de `.field`	Evitá el select con JS: no hace falta
Chips de antigüedad	`<input type="radio">` + `<label>` con Tailwind	Mismo `name` = opción única
Cards de plan	`<label>` que envuelve un `.card` + radio	Toda la card es clickeable
Badge "Recomendado"	`<span class="badge">`	Variantes con `data-variant`
Resumen	`<div class="card">` con `<header>` y `<section>`	Card lateral en desktop
Aviso de simulación	`<div class="alert">`	`data-variant="destructive"` en error
Checkbox términos	`<input type="checkbox">` dentro de `.field`	
Stepper	Tailwind a mano	Recibe el paso actual y pinta 3 estados

Ejemplo de campo con error, tal como se ve en la pantalla 04:

```
<div role="group" class="field" data-invalid>
  <label for="email">Email</label>
  <input id="email" type="email" aria-invalid="true"
         aria-describedby="email-error" value="juan.perez@">
  <p id="email-error" role="alert">Ingresá un email válido</p>
</div>
```

8. Responsive: mobile y desktop

Trabajá **mobile first**: primero la versión de 390 px y después agregá los cambios para pantallas grandes con el prefijo `lg:` de Tailwind.

Elemento	Mobile (390)	Desktop (1440, desde lg:)
Layout checkout	Una columna	Dos columnas: formulario + resumen de 380 px
Resumen	Barra fija arriba del botón	Card lateral
Campos de formulario	Uno por fila (salvo pares cortos)	2 o 3 por fila
Planes	Apilados	Tres en fila
Botones de acción	Ancho completo en el footer	"Volver" a la izquierda, "Continuar" a la derecha
Inicio	Grilla 2×2 de productos	Hero a la izquierda y grilla a la derecha

```
<div class="grid gap-8 lg:grid-cols-[1fr_380px]">
  <form class="card"> ... </form>
  <aside class="card"> ... resumen ... </aside>
</div>
```

> **Regla de oro**
> El resumen se genera **una sola vez** desde el estado con `renderResumen()`. Si lo duplicás para mobile y desktop, duplicás errores.

9. Hitos y entregables

Hito	Objetivo	Se entrega
H1 · Maquetado	Las 7 pantallas en HTML + Basecoat, estáticas, mobile y desktop	Repo con `index.html` y capturas
H2 · Navegación y estado	Objeto `solicitud`, `render()`, stepper y "Volver"	Flujo navegable de punta a punta
H3 · Validaciones y cálculo	Validaciones, botones deshabilitados, `planes.json` y cuotas	Los 3 casos de prueba dan bien
H4 · Pago y cierre	Pago simulado, éxito/error, póliza, README y deploy	URL publicada + README

Licenciatura en Sistemas de Información

Diseño UXUI - 2026

Trabajo Práctico

Qué entregar

- Repositorio en GitHub con commits por hito (mensajes claros, no "cambios").
- Sitio publicado (GitHub Pages, Netlify o Firebase Hosting).
- `README.md` con: cómo correrlo, decisiones tomadas, capturas mobile y desktop, y qué bonus hiciste.

10. Tips clave

1. **El estado manda.** Cada evento modifica `solicitud` y después llama a `render()`. Si algo se ve mal, mirá primero el estado con `console.table(solicitud)`.
2. **Números como números.** Los inputs devuelven strings. Convertí con `Number()` y limpiá los separadores antes de calcular: "600.000" no es un número.
3. **Formato de moneda.** Usá `new Intl.NumberFormat("es-AR", { style: "currency", currency: "ARS", maximumFractionDigits: 0 })`. No armes el string a mano.
4. **Validaciones como funciones puras.** `validarEmail(valor)` devuelve un mensaje o `null`. Se prueban solas y se reutilizan.
5. **Validá en el momento justo.** Mostrá el error en `blur` (al salir del campo), no mientras la persona escribe la primera letra. Revalidá todo al intentar avanzar.
6. **Un botón deshabilitado necesita una explicación.** Mostrá qué falta, como en el diseño de la pantalla 04.
7. **Usá el atributo hidden en lugar de borrar HTML.** Ocultar y mostrar secciones es más simple que crearlas y destruirlas con JavaScript.
8. **Accesibilidad mínima.** Cada input con su `<label for>`, errores con `aria-invalid` y `role="alert"`, y foco al primer campo con error al intentar avanzar.
9. **Evitá el doble click en "Pagar".** Deshabilitá el botón apenas se hace click y volvé a habilitarlo solo si el pago falla.
10. **Commits chicos.** Un commit por pantalla o funcionalidad. Si algo se rompe, volver atrás es fácil.
11. **Copiá del Figma, no de memoria.** Espaciados, tamaños y colores ya están definidos: inspeccionalos.

11. Errores comunes

Error	Por qué pasa	Cómo evitarlo
La cuota da NaN	El valor viene como "$ 600.000"	Limpiar con `replace(/\D/g, "")` y `Number()`
Al volver atrás se borran los datos	Se regenera el HTML o se resetea el formulario	Rellenar los inputs desde el estado en `render()`

Error	Por qué pasa	Cómo evitarlo
Se pueden elegir dos planes	Radios con distinto `name`	Mismo `name="plan"` en los tres
El formulario recarga la página	Submit por defecto del `<form>`	`event.preventDefault()`
El fetch del JSON falla	Abrir el HTML con doble click (file://)	Usar Live Server o `npx serve`
Pago doble	Botón habilitado durante el proceso	Deshabilitar al hacer click
Precio distinto en resumen y planes	Se calcula en dos lugares	Una sola función `calcularCuota()`

12. Bonus (opcionales)

- **Persistencia:** guardar `solicitud` en `localStorage` para que sobreviva a un F5 (y limpiarla al terminar).
- **Editar desde el resumen:** links "Editar" que llevan al paso correspondiente y vuelven al pago.
- **Toast** de Basecoat al confirmar el pago (cargando solo el JS de toast).
- **Habilitar otro producto** (Bicicleta o Viaje) reutilizando el mismo flujo con otros campos y otra tabla de tasas.
- **Máscaras de input:** tarjeta en grupos de 4 y vencimiento con barra automática.
- **Modo oscuro** usando las variables de Basecoat.
- **Setup con Vite** (opción 3.3) y deploy automático con GitHub Actions.

Los bonus suman hasta **10 puntos extra** (sin superar la nota máxima).

13. Checklist antes de entregar

- ☐ Las 7 pantallas existen y se ven bien en 390 px y en 1440 px.
- ☐ Los 3 casos de prueba de la sección 5.1 dan exactamente el valor esperado.
- ☐ No se puede avanzar con datos inválidos y cada error tiene su mensaje.
- ☐ "Volver" conserva los datos en todos los pasos.
- ☐ La tarjeta 4000 0000 0000 0002 lleva al error y "Reintentar" vuelve al pago con todo cargado.
- ☐ La póliza tiene formato POL-AAAA-NNNNNN.
- ☐ No hay errores en la consola del navegador.
- ☐ El sitio está publicado y el README tiene capturas y la URL.